

Java

Le interfacce

Interfacce

- Nella sua parte non statica, una classe fornisce la *definizione di un ADT*
 - parte visibile esternamente (public)
 - *implementazione*
 - dati privati, protetti, o visibili nel package
 - metodi privati, protetti, o visibili nel package
- In questo modo, però, diventa impossibile *dare una pura specifica di interazione di un ADT, senza definirlo.*

Le interfacce

- ❑ Permettono di definire dei tipi in modo astratto
- ❑ Non sono dotate di implementazione
- ❑ Non possono essere istanziate
- ❑ Una classe può implementare una o più interfacce
- ❑ Definiscono nomi di tipi così come le classi

**Una interfaccia è legata al solo processo di progettazione;
una classe è un misto di progettazione e implementazione**

Che cos'è un'interfaccia?

- Un'interfaccia è una collezione di metodi headers (senza la loro definizione).
- Un'interfaccia può anche dichiarare costanti.
- Appaiono simili alle classi astratte ma hanno significative differenze:
 - Un'interfaccia non può avere metodi con l'implementazione
 - Una classe può implementare molte interfacce ma può essere derivata da una sola superclasse
 - Le interfacce non fanno parte della gerarchia delle classi, così che classi non discendenti l'una dall'altra possono implementare la stessa interfaccia

Scopo:

Definire un protocollo del comportamento che deve essere fornito ad una classe. Una qualsiasi classe che implementa una data interfaccia è quindi obbligata a fornire l'implementazione di tutti i metodi elencati nell'interfaccia..

Interfacce e progetto

- Può essere utile disporre di un *nuovo costrutto* simile alla (parte non statica di una) classe, nel senso di consentire la definizione del "modo di interagire" di un'entità...

ma non tenuto a fornire *implementazioni*...

... né legato alla *gerarchia di ereditarietà* delle classi, con i relativi vincoli.

- Le interfacce inducono un diverso *modo di concepire il progetto*
 - prima si definiscono le interfacce delle entità che costituiscono il sistema
 - in questa fase si giocano *scelte di progetto (pulizia concettuale)*
 - poi si realizzeranno le classi che “implementeranno” tali interfacce
 - in questa fase entreranno in gioco *scelte implementative (efficienza ed efficacia)*

Dichiarazione di Interfacce

- Una *interfaccia* costituisce una *pura specifica di interazione*
 - contiene *solo dichiarazioni* di metodi
 - ed eventualmente costanti
 - ma non variabili né *definizioni* di metodi
- Praticamente, una *interfaccia*
 - è strutturalmente *analoga a una classe...*
 - ma è introdotta dalla parola chiave `interface` anziché `class` e contiene solo *dichiarazioni* di metodi

Dichiarazione di Interface

```
<modificatore> interface <nome_interfaccia>  
    [ extends <lista_interfacce> ]  
{ <corpo_interfaccia> }
```

Dichiarazione di interface

```
<modificatore> interface <nome_interfaccia>  
[ extends <lista_interfacce> ]  
{ <corpo_interfaccia> }
```



- Annotazioni
- public (se manca public la visibilità è package)
- abstract (tale modificatore è omesso)
- strictfp

Dichiarazione di interface

```
<modificatore> interface <nome_interfaccia>  
    [ extends <lista_interfacce> ]  
{ <corpo interfaccia> }
```

- Costanti di interfaccia
 - Consente di definire attributi che sono implicitamente public, final e static
 - Non e consentito il blank final
- Metodi
 - Sono implicitamente abstract
 - Non sono ammessi modificatori eccetto le annotazioni
 - Il corpo di ciascun metodo è costituito dal solo “;”
 - I metodi che li implementano possono avere un qualsiasi modificatore

Esempi

- Definizione dell'astrazione "Collezione"
 - Cosa si intende per "Collezione"?
 - *Come ci si aspetta di poter interagire* con un'entità qualificata come "Collezione"?
- *Indipendentemente da qualsiasi scelta o aspetto implementativo,* una "Collezione" è tale perché
 - è un contenitore
 - è possibile chiedersi se è vuota e quanti elementi contiene
 - vi si possono aggiungere e togliere elementi
 - è possibile chiedersi se un elemento è presente o no ...

Esempio

- Una "Collezione" è dunque una *qualsiasi entità che si conformi a questo "protocollo di accesso"*
 - è possibile chiedersi se è vuota
 - è possibile chiedersi quanti elementi contiene
 - vi si possono aggiungere e togliere elementi
 - è possibile chiedersi se un elemento è presente o no
 - ...
- Si definiscono così astrazioni di dato in termini di *comportamento osservabile*, ossia di
 - “cosa ci si aspetta” da esse
 - “cosa si pretende che esse sappiano fare”
 - rinviando a tempi successivi la realizzazione pratica di ADT (classi) che rispettino questa specifica.

Esempio

```
public interface Collection {  
    public boolean add(Object x);  
    public boolean contains(Object x);  
    public boolean remove(Object x);  
    public boolean isEmpty();  
    public int size();  
  
    ...  
}
```

Gerarchie di interfacce

- ▣ Le interfacce possono dare luogo a *gerarchie*, proprio come le classi:

```
public interface List extends Collection  
{ ... }
```

La gerarchia delle interfacce:

1. è una gerarchia separata da quella delle classi
2. è slegata dagli aspetti implementativi
3. esprime le *relazioni concettuali* della realtà
4. guida il progetto del *modello della realtà*.

Gerarchie: esempio

- Come in ogni gerarchia, anche qui le interfacce derivate:
 - possono aggiungere nuove *dichiarazioni di metodi*
 - possono aggiungere nuove *costanti*
 - *non possono* eliminare nulla
- Significato: “*Ogni lista è anche una collezione*”
 - ogni lista può interagire col mondo come farebbe una collezione (*magari in modo specializzato*)...
 - ... ma può avere *proprietà peculiari al concetto di lista*, che non valgono per una “collezione” qualsiasi.

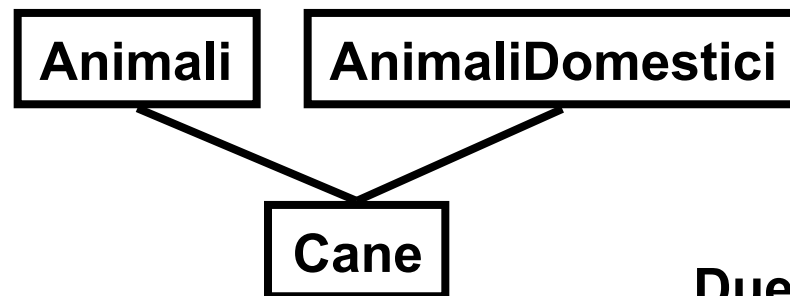
Gerarchie: esempio

- Ad esempio, una “Lista” ha un concetto di *sequenza*, di *ordine* fra i suoi elementi
 - esiste quindi un *primo elemento*, un secondo, ...
 - quando si aggiungono nuovi elementi bisogna dire *dove* aggiungerli (ad esempio, in coda)
 - è possibile recuperare un elemento *a partire dalla sua posizione* (il primo, il decimo,...)

```
public interface List extends Collection {  
    public boolean add(int posizione, Object x);  
    public Object get(int posizione);  
  
    ...  
}
```

No Ereditarietà Multipla in Java

Alcuni linguaggi permettono l'ereditarietà multipla



Due padri per un figlio!!!

L'ereditarietà multipla fornisce una grande flessibilità ...ma anche una grande confusione

Ereditarietà multipla

- *Java la supporta fra interfacce*
 - una interfaccia contiene *solo dichiarazioni* di metodi
 - non ha implementazioni e nessun problema di collisione fra *metodi* omonimi
 - non ha variabili e nessun problema di collisione fra *dati* omonimi
- *È un potente strumento di modellizzazione*

Ereditarietà multipla: esempio

```
public interface Lavoratore {  
    ...  
}  
public interface Studente {  
    ...  
}  
public interface StudenteLavoratore extends  
    Lavoratore, Studente {  
    ...  
}
```

■ Dopo **extends** può esservi un elenco di *più interfacce*

Dall'astrazione all'implementazione

- Un'interfaccia definisce una astrazione di dato in termini di *comportamento osservabile*.
- Per sua natura, però, *non implementa nulla*.
- Qualcuno dovrà prima o poi *implementare la astrazione* definita dall'interfaccia.
- A questo fine, una *classe* può *implementare (una o più) interfacce*
 - Le interfacce contengono *dichiarazioni* di metodi
 - La classe *definisce (implementa)* tutti i metodi delle interfacce che si impegna a implementare.

Dall'astrazione all'implementazione

- Essendosi impegnata a implementare tale interfaccia, la classe `LinkedList` DEVE definire TUTTI i metodi dichiarati dall'interfaccia `List`
- Se non lo fa, il compilatore Java segnala ERRORE.

```
public class LinkedList implements List{  
    ...  
}
```

Dall'astrazione all'implementazione

- Una classe può implementare *più interfacce*.
- Essendosi impegnata a implementare entrambe le interfacce, la classe `myList` DEVE definire TUTTI i metodi dichiarati dall'interfaccia `List` più TUTTI quelli dichiarati dall'interfaccia `Comparable`
- Se non lo fa, il compilatore Java segnala ERRORE.

```
public class myList implements List, Comparable {  
    ..  
}
```

Interfacce: uso

- Il nome di una interfaccia può essere usato come *identificatore di tipo* per *riferimenti* e *parametri formali* di metodi e funzioni.

```
public static void main(String s[]) {  
    Collection c;  
    ...  
    List l1;  
    List l2;  
    ...  
}
```

Interfacce: uso

- A tali riferimenti si possono assegnare *istanze di una qualunque classe che implementi l'interfaccia*
- Le classi `Vector` e `LinkedList` implementano entrambe l'interfaccia `List`, che a sua volta estende `Collection`
- Il costruttore di default di `Vector` e `LinkedList` crea una collezione di oggetti vuota

```
public static void main(String s[]){  
    Collection c = new Vector();  
    c.add(new Point(3,4));  
    .  
    ..  
    List l1 = new LinkedList();  
    List l2 = new LinkedList(c);  
}
```

Interfacce: uso

- In effetti, quando una classe implementa una interfaccia, le *dichiarazioni dei metodi* contengono *il nome dell'interfaccia...*

```
public List copy(List z) {  
    return new LinkedList(z);  
}
```

- ... ma in realtà tali metodi ricevono e *manipolano istanze di una qualche classe* che implementi l'interfaccia richiesta.

Interfacce: uso

- I riferimenti che introduciamo devono *riflettere la vista esterna*
 - `Collection`, `List`, ...
- Gli oggetti che creiamo devono necessariamente essere istanze di *classi concrete*
 - `LinkedList`, `Vector`, ...
- Esempi

```
Collection c = new Vector();
```

```
List l1 = new LinkedList();
```

```
List l2 = new LinkedList(c);
```

Alcune interfacce del package standard

- **Cloneable**: oggetti di classi che implementano tale interfaccia supportano la clonazione
- **Comparable**: oggetti di classi che implementano tale interfaccia hanno una relazione di ordinamento che permette di confrontarli gli uni con gli altri
- **Runnable**: oggetti di classi che implementano tale interfaccia rappresentano unità eseguibili in un proprio thread
- **Serializable**: oggetti di classi che implementano tale interfaccia possono essere scritti sotto forma di flusso di byte

Interfacce marker

- Sono interfacce che hanno un corpo vuoto: né metodi né costanti
- Non dichiarando alcun metodo si limitano soltanto a identificare che le classe che la implementano sono dotate di una qualche proprietà generale
- Esempio:
 - L'interfaccia **clonable**